



Programming Fundamentals

By

Dr Suhail Asghar

Dawood University Of Engineering &
Technology Karachi

Functions

- When writing programs, we often need to perform certain tasks multiple times.
- Instead of writing the same code again and again, we group that code into a reusable block.
- This reusable block is called a Function.
- Definition: A Function is a block of code that performs a specific task and can be reused whenever required.

Why
Functions
are
Needed

Without Functions	With Functions
Code repetition occurs	Code becomes reusable
Difficult to debug	Easy to maintain and debug
Program becomes very long	Program becomes organized and structured
Hard to understand	Improves readability

Function parts

- A function generally has:

1. Function Name

2. Parameters (optional)

3. Body (statements)

4. Return Statement (optional)

```
return_type  
function_name(parameter_list) {  
    // body  
    return value; // if  
return_type != void  
}
```



Parameters, scope, and advanced features

- Goals: parameter passing (value/ref/pointer),
 - Default args,
 - Overloading,
 - Recursion,
 - Inline/constexpr/static,
 - Templates,
 - Lambdas.
- 
- 
- 
- 

Return Type

- Return Type: This tells what type of value the function will return to the part of the program that called it.
- If the function returns nothing, the return type is void
- If the function returns an integer, the return type is int
- If it returns decimal value → float or double
- If it returns a character → char

- Void Function

- A function whose return type is void does not return a value to the calling program.

```
void  
functionName(parameter_list);
```

- Non-Void Function / Value-Returning Function

- A function whose return type is int (or any datatype) must return a value using the return statement.



Declaration (prototype) vs Definition

- Declaration (prototype) vs Definition

```
int add(int a, int b); // prototype
```

- Definition

```
int add(int a, int b) { return a + b; }
```



Default arguments

- `int multiply(int a, int b = 2) { return a * b; }`
- `// multiply(3) -> 6,`
`multiply(3,4) -> 12`

- Default args must be specified in declaration or definition (but not both inconsistently).

Wrong Example (Default Arguments in Both)

```
int multiply(int a, int b = 2);  
// declaration with default  
  
int multiply(int a, int b) {  
// definition without default  
    return a * b;  
}
```

A separate declaration is needed when the function is used BEFORE its definition

```
#include <iostream>
using namespace std;
// Function definition (no
// declaration needed)
int add(int a, int b) {
    return a + b;
}
int main() {
    cout << add(2, 3);
}
```

```
#include <iostream>
using namespace std;
// ---- Declaration ----
int add(int a, int b);
int main() {
    cout << add(2, 3);    //
    OK
}
// ---- Definition ----
int add(int a, int b) {
    return a + b;
}
```

The declaration lets the compiler know: "Don't worry, this function exists – you'll see the body"

Example - print and return

```
#include <iostream>
using namespace std;

int square_return(int n) {
    return n * n;
}

void square_print(int n) {
    cout << n * n << '\n';
}
```

```
int main() {
    cout <<
    "square_return(5) = " <<
    square_return(5) << '\n';
    square_print(7); //
    prints 49
    return 0;
}
```

Example — add

```
int add(int a, int b) { return a + b; }
```

- Calling a function

```
int x = add(3,4); // x = 7
```



Return vs cout (print)

- return passes value to caller.
 - cout prints to console.
 - Prefer return when result is needed for further computation.
- 

Parameter Passing: value vs reference vs pointer

1. Pass by value (copy)

```
void increment_by_value(int n) {  
    n = n + 1; // only local copy  
    changes}
```

2. Pass by reference (modifies caller)

```
void increment_by_ref(int &n) {  
    n = n + 1; // original variable  
    modified}
```

3. Pass by pointer (C-style)

```
void increment_by_ptr(int *p) {  
    if (p) *p = *p + 1;}
```

Example showing all three:

```
#include <iostream>  
using namespace std;  
void pv(int n) { n++; }  
void pr(int &n) { n++; }  
void pp(int *n) { if(n) (*n)++; }  
int main() {  
    int a = 5;  
    pv(a); cout << "after pv: " << a <<  
    '\n'; // 5  
    pr(a); cout << "after pr: " << a <<  
    '\n'; // 6  
    pp(&a); cout << "after pp: " << a <<  
    '\n'; // 7  
}
```

Local vs Global variables

- **Local:** defined inside function – lifetime is function call.
- **Global:** defined outside all functions – visible (unless restricted) across file.
- Prefer minimizing globals.

```
#include <iostream>
using namespace std;
int g = 100; // Global variable

void test() {
    int g = 50; // Local variable shadows
    global variable
    cout << g << endl;    // prints 50 (local)
}

int main() {
    cout << g << endl;    // prints 100 (global)
    test();
    cout << g << endl;    // prints 100 (global
    remains unchanged)    }
```

Function overloading

- Function Overloading means having more than one function with the same name but different parameters.
- In C++, the compiler decides which function to run based on:
 1. Number of parameters,
 2. Type of parameters or
 3. Type of parameters
- Function overloading is a feature of C++ where multiple functions can have the same name but different number or type of parameters.
- This allows the same function name to perform different tasks.
- The compiler decides which function to call based on the argument

Example 1: Same name,
different parameter lists

```
int area(int side) { return side *  
side; }           // square
```

```
int area(int l, int w) { return l * w;  
}                     // rectangle
```

```
double area(double r) { return  
3.1415926535 * r * r; } // circle  
(radius)
```

Example 2: Same Name, Different Number of Arguments

```
#include <iostream>
using namespace std;
void show(int a) {
    cout << "Value = " <<
a << endl;
}
void show(int a, int b) {
// same function name,
different parameters
    cout << "Sum = " << a
+ b << endl;
}
```

```
int main() {
    show(5);          //
calls first function
    show(5, 10);      //
calls second function
    return 0;
}
```

Example 3: Same Name, Different Data Types

```
#include <iostream>
using namespace std;
int add(int a, int b) {
    return a + b;
}
double add(double a,
double b) {    // same
name, different type
    return a + b;
}
```

```
int main() {
    cout << add(3, 4)
<< endl;    // calls
int version
    cout << add(2.5,
3.6) << endl; // calls
float version
    return 0;
}
```

Function Overload ing (Summary)



1. Number of arguments



2. Types of arguments



3. Order of arguments

Function to Display Table of a Number

```
#include <iostream>
using namespace std;
void Table(int n) {
    for(int i = 1; i <=
10; i++) {
        cout << n << " x "
<< i << " = " << n * i <<
endl;
    }
}
```

```
int main() {
    int num;
    cout << "Enter a
number: ";
    cin >> num;

    Table(num);    //
function call
}
```

Factorial Using Function (Without Recursion)

```
#include <iostream>
using namespace std;
int factorial(int n) {      //
function definition
    int fact = 1;
    for(int i = 1; i <= n; i++)
    {
        fact = fact * i;
    }
    return fact;
}
```

```
int main() {
    int num;
    cout << "Enter a number: ";
    cin >> num;
    cout << "Factorial = " <<
factorial(num);
    return 0;
}
```

Area of shapes (overloading + default args)

```
#include <iostream>
using namespace std;

double area(double r) { return
3.141592653589793 * r * r; }

int area(int l, int w) { return l * w; }
int area(int side) { return side * side;
}

int main() {
    cout << area(5.0) << '\n';    //
circle
    cout << area(4, 3) << '\n';    //
rectangle
    cout << area(6) << '\n';        // square
(ambiguous with int area(side) - ok)
}
```

Reverse number and palindrome (function)

```
#include <iostream>
```

```
using namespace std;
```

```
int reverseNumber(int n) {  
    int rev = 0;  
    while(n > 0) {  
        rev = rev * 10 +  
(n % 10); // take last  
digit and add to reverse  
        n = n / 10;  
// remove last digit  
    }  
    return rev;  
}
```

```
int main() {  
    int num;  
    cout << "Enter a  
number: ";  
    cin >> num;  
  
    cout << "Reversed  
Number = " <<  
reverseNumber(num) <<  
endl;  
}
```

Prime check

```
#include <iostream>

using namespace std;

bool isPrime(int n) { // Function
to check if number is prime
    if(n <= 1)
        return false; //
numbers <=1 are not prime
    for(int i = 2; i * i <= n;
i++) {
        if(n % i == 0)
            return false; //
divisible → not prime
    }
    return true; //
no divisor → prime
```

```
int main() {
    int num;
    cout << "Enter a number:
";
    cin >> num;
    if(isPrime(num))
        cout << num << " is a
Prime Number.";
    else
        cout << num << " is
NOT a Prime Number.";
}
```

Full example program – small calculator using functions

```
int main() { int a, b; char op;
    cout << "Enter: a op b (e.g. 4 + 5): ";
    cin >> a >> op >> b;
    switch(op) {
        case '+':
            cout <<
            add(a,b) << '\n';
            break;
        case '-':
            cout <<
            sub(a,b) << '\n';
            break;
        case '*':
            cout <<
            mul(a,b) << '\n';
            break;
        case '/':
            if (b ==
            0) {
                cout << "Division by 0
                is not possible\n";
            }
    }
}
```

#include <iostream>

using namespace std;

int add(int a, int b){ return a+b; }

int sub(int a, int b){ return a-b; }

int mul(int a, int b){ return a*b; }

double divide(int a, int b){ return b ? static_cast<double>(a)/b : 0.0; }

Recursion

- Function calls itself.
Example: factorial

```
long long factorial(int n)
{
    if (n <= 1) return 1;
    return n * factorial(n
- 1);
}
```

Warning: watch stack depth.

Factorial Using Recursion

```
#include <iostream>
using namespace std;

int factorial(int n) {
    if(n == 0 || n == 1)
        return 1;
    return n * factorial(n
- 1);    // recursive call
}
```

```
int main() {
    int num;
    cout << "Enter a
number: ";
    cin >> num;
    cout << "Factorial
= " << factorial(num);
    return 0;
}
```

Print Numbers from 1 to N Using Recursion

```
#include <iostream>
using namespace std;
void printNumbers(int n)
{
    if(n == 0)    // Base
case
        return;
    printNumbers(n - 1);
// Recursive call
    cout << n << " ";
}

int main() {
    printNumbers(5);
}
```

Sum of First N Numbers Using Recursion

```
#include <iostream>
using namespace std;

int sum(int n) {
    if(n == 0)    // Base
case
        return 0;
    return n + sum(n - 1);
// Recursive call
}
```

```
int main() {
    int n;
    cout << "Enter n: ";
    cin >> n;
    cout << "Sum = " <<
sum(n);
}
```